

Unidad 1

Análisis semántico

¿El programa **tiene sentido** además de estar bien escrito?

Prerrequisito: parser y AST de LA I

Glosario Unidad 1

Término	Significado
AST	Árbol con la estructura del programa (viene del parser)
Tabla de símbolos	Directorio: nombre → tipo, ámbito, línea
Ámbito (scope)	Dónde “vive” una variable: global, local, parámetro
Tipo	<code>int</code> , <code>float</code> , <code>bool</code> — qué operaciones están permitidas
Acción semántica	Código que corre cuando el parser reconoce una regla
Visitor	Función que recorre el AST nodo por nodo

Semántico = fase 3 del pipeline. Responde: *¿las variables existen y los tipos cuadran?*

¿Dónde estamos?

✓ Léxico (LA I – tokens)
✓ Sintáctico (LA I – AST)
→ SEMÁNTICO ← ESTA UNIDAD
IR (U2)
Optimización (U3)
Codegen (U4)

Entrada	AST del parser	
Salida	Errores con número de línea ● AST validado + tabla de símbolos	

¿Qué NO hace el semántico?

Fase	Lo que hace	¿Semántico?
Léxico	Separa tokens	✗
Sintáctico	Construye AST, valida <code>;</code> y paréntesis	✗
Semántico	Tipos, variables declaradas, ámbitos	✓
IR / ASM	Genera instrucciones	✗ (unidades 2–4)

Pregunta clave del semántico: «¿**Tiene sentido este programa ya bien formado?**»

Sintáctico vs semántico — ejemplos

```
int x = ;           // SINTÁCTICO: falta expresión después de =
```

```
int x;  
x = y + 1;         // SEMÁNTICO: y no fue declarada
```

```
int x;  
bool b;  
x = b;             // SEMÁNTICO: no puedo guardar bool en int
```

El **parser** acepta `x = y + 1` porque la **forma** es válida.

El **semántico** rechaza porque `y` **no está en la tabla de símbolos**.

AST — repaso visual

```
int x = 3 + 4 * 2;
```

```
Decl(tipo=int, nombre=x)  
├─ Assign  
│  ├─ Id(x)  
│  └─ Add( Num(3), Mul( Num(4), Num(2) ) )
```

El semántico **visita** cada nodo:

«¿**x** existe? ¿**3** y **4** son compatibles con *****? ¿el resultado es **int**?»

Tabla de símbolos

Como un **directorio telefónico** del programa:

Nombre	Tipo	Ámbito	Línea
x	int	global	1
f	función → int	global	3
a	int	parámetro de f	3

Operaciones típicas:

`insert` (declarar), `lookup` (buscar), `enter_scope` / `exit_scope` (entrar/salir de función o bloque)

Ámbitos (scopes)

```
int g;           // ámbito global
int f(int a) {   // enter_scope → función f
    int b;       // local de f
    { int c; }    // bloque interno (opcional)
}               // exit_scope
```

Implementación: pila de diccionarios — el tope es el ámbito **activo**.

Buscar **a** dentro de **f** → primero en local; si no, subir niveles.

Reglas de tipos — MiniLang

Operación	¿Qué operandos?	Resultado
<code>+ - * /</code>	dos <code>int</code>	<code>int</code>
<code>+ - * /</code>	dos <code>float</code>	<code>float</code>
<code>< > ==</code>	dos <code>int</code>	<code>bool</code>
<code>=</code> asignación	mismos tipos izq/der	OK

```
int x; bool b; x = b;    // ERROR semántico
```

Acciones semánticas

Producción gramatical = regla de forma (BNF).

Acción semántica = código que ejecutas al aplicar esa regla.

```
decl → type ID ';'      { tabla.insert(ID, type); }
```

Cuando el parser reduce «declaración de variable», **insertas** en la tabla.

Producción = qué forma tiene. **Acción** = qué efecto tiene en el significado.

Pila semántica (simulación)

Al procesar `3 + 4`:

Paso	Acción	Pila (tope a la derecha)
1	leer <code>3</code>	... int, 3
2	leer <code>4</code>	... int, 3, int, 4
3	operador <code>+</code>	... int, 7

Útil en examen: simular 5–6 pasos en una tabla.

Formato de errores (obligatorio)

```
Error semántico línea 5: variable 'y' no declarada  
Error semántico línea 8: no se puede asignar bool a int
```

Siempre: **número de línea** + **causa clara**.

✗ Mal: «error en expresión»

✓ Bien: «variable 'y' no declarada en línea 5»

Ejemplo correcto — paso a paso

```
int a;  
int b;  
b = a + 1;
```

1. Declarar `a`, `b` en tabla (tipo `int`, global)
2. Buscar `a` → tipo `int`
3. Literal `1` → tipo `int`
4. `+` con dos `int` → resultado `int`
5. Asignar a `b` (`int`) → **OK**

Ejemplo con error

```
int x;  
x = true;
```

1. `x` está declarado como `int`
2. `true` es `bool`
3. Asignación `int = bool` → **ERROR semántico línea 2**

Implementación sugerida

```
compilador/  
  parser/          ← LA I (no modificar gramática)  
  semantic/  
    symbol_table.py  
    analyzer.py  
  main.py          ← parser → semantic → imprimir errores
```


Visitor sobre el AST

```
def analyze(nodo, tabla):  
    if nodo es Decl:  
        tabla.declare(nombre, tipo)  
    elif nodo es Assign:  
        verificar_tipos(izquierda, derecha)  
    elif nodo es BinOp:  
        return tipo_del_resultado(hijo_izq, hijo_der, operador)  
    # recursión en hijos...
```

Visitor = patrón «visitar» cada nodo del árbol recursivamente.

Casos de prueba obligatorios

#	Programa	¿Error?	Motivo
1	<code>int x; x=1;</code>	No	OK
2	<code>x=1;</code>	Sí	<code>x</code> no declarado
3	<code>int x; int x;</code>	Sí	redeclaración
4	<code>int x; bool b; x=b;</code>	Sí	tipos
5	<code>bool b; b=1+true;</code>	Sí	<code>+</code> no aplica a bool

Tareas de la unidad

Tarea	Puntos	Entrega
Exposición	40	Diapositivas + oral (tema del temario)
Avance compilador	60	ZIP con <code>semantic/</code> + pruebas
Examen	100	En línea

Repaso — 5 ideas clave

1. Semántico = **significado**, no forma gramatical
2. **AST + tabla de símbolos + tipos**
3. Diferencia clara: sintáctico vs semántico
4. Errores con **línea y causa**
5. Confía en el parser de LA I — tú validas **sentido**
Próxima unidad: AST → triplos (**IR**)